

Condicionais em Python

Decidindo com `if`, `elif` e `else`

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
`andreyoshiaki@dcomp.ufs.br`

O problema da triagem

Como o computador decide

A ordem importa

Sua vez

O problema da triagem

A situação

Chegam dezenas de pacientes por hora. Alguém precisa decidir, para *cada um*, quem é atendido primeiro.

A decisão não pode depender do humor de quem está na recepção: dois pacientes com os mesmos dados têm de receber a mesma classificação.

O que o computador precisa

Uma regra escrita de forma que a máquina consiga **seguir**: dados de entrada, testes, uma resposta.

Nossa ferramenta de hoje

O comando `if` — e os seus dois companheiros.

Cada paciente tem dois dados: **pressão sistólica** (real, na escala mmHg/10 — o valor 14.0 representa 140 mmHg) e **idade** (inteiro, em anos).

Classificação	Descrição
NORMAL	pressão e idade dentro dos limites saudáveis
BAIXO	pressão acima de 120 mmHg, sem outros agravantes
MEDIO	pressão acima de 140 mmHg ou idade acima de 60
ALTO	pressão acima de 140 mmHg e idade acima de 60
CRITICO	pressão inferior a 90 mmHg — emergência, qualquer idade

O aviso do enunciado

“Um mesmo paciente pode se enquadrar em mais de uma descrição; cabe a você determinar a **ordem correta de verificação**.”

Nome	Pressão	Idade	Em mmHg
Antonio	15.0	70	150 mmHg
Beatriz	14.5	30	145 mmHg
Cesar	12.5	80	125 mmHg
Daniela	11.0	20	110 mmHg
Eduardo	8.5	45	85 mmHg

Faça agora, no papel

Classifique os cinco pacientes usando a tabela do slide anterior. Guarde a sua folha — vamos comparar com o computador.

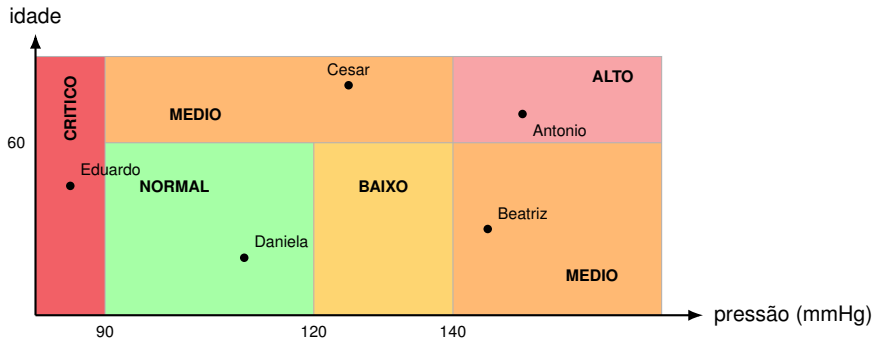
Antonio: 150 mmHg, 70 anos. Confira as descrições:

Descrição	Serve para o Antonio?	Por quê
BAIXO	sim	150 está acima de 120
MEDIO	sim	150 está acima de 140
ALTO	sim	150 acima de 140 e 70 acima de 60

A pergunta central da aula

Três descrições verdadeiras ao mesmo tempo. O computador vai escolher **uma**. Quem decide qual?

Resposta: quem escreveu o código — pela **ordem** em que colocou os testes.



Cada ponto é um paciente. Cada região, uma classificação. Repare: o Antonio está no canto onde *pressão alta* e *idade alta* se encontram.

Antes de rodar: qual será a saída?



```
1 def risco(pressao, idade):
2     if pressao < 9.0:
3         return "CRITICO"
4     elif pressao > 14.0 and idade > 60:
5         return "ALTO"
6     elif pressao > 14.0 or idade > 60:
7         return "MEDIO"
8     elif pressao > 12.0:
9         return "BAIXO"
10    else:
11        return "NORMAL"
12
13 print("Antonio:", risco(15.0, 70))
14 print("Beatriz:", risco(14.5, 30))
15 print("Cesar:", risco(12.5, 80))
16 print("Daniela:", risco(11.0, 20))
17 print("Eduardo:", risco(8.5, 45))
```

Escreva a sua previsão

Antonio: _____

Beatriz: _____

Cesar: _____

Daniela: _____

Eduardo: _____

Regra do jogo

Escreva **antes** de rodar. Errar a previsão é o momento em que se aprende mais.

Como o computador decide

O que o Python imprimiu

```
Antonio: ALTO  
Beatriz: MEDIO  
Cesar: MEDIO  
Daniela: NORMAL  
Eduardo: CRITICO
```

Compare com a sua folha

- Acertou os cinco? Ótimo — agora explique *por que* o Antonio é ALTO e não MEDIO.
- Errou algum? Marque qual. Vamos descobrir exatamente em que linha o computador decidiu.

Onde mora a decisão

A resposta está nas quatro palavras `if`, `elif`, `or`, `and` — e na ordem delas.

palavra-chave condição (dá True ou False) dois-pontos



```
if pressao < 9.0:
```

```
    return "CRITICO"
```

bloco: só roda se a condição for True

indentação
(4 espaços)

Em Python a indentação não é enfeite

Ela é que diz o que está dentro do `if`. Perder um espaço muda o programa.

Toda condição é uma expressão que vale True ou False.

Operador	Lê-se
<	menor que
<=	menor ou igual a
>	maior que
>=	maior ou igual a
==	é igual a
!=	é diferente de

```
1 >>> pressao = 15.0
2 >>> pressao < 9.0
3 False
4 >>> pressao > 14.0
5 True
6 >>> idade = 70
7 >>> idade > 60
8 True
```

Ideia-chave

O `if` não entende “pressão alta”. Ele só sabe olhar para um `True` ou um `False`.

= guarda um valor

```
1 idade = 70    # agora idade vale 70
```

== faz uma pergunta

```
1 idade == 70   # True
```

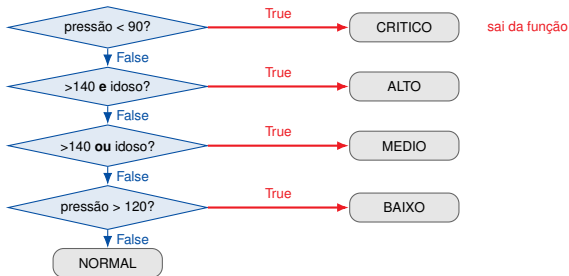
Erro que o Python recusa

```
1 if idade = 70:  
2     print("idoso")
```

SyntaxError: invalid syntax

Como não esquecer

Leia em voz alta: = é “recebe”, == é “é igual a?”.



A cascata é uma **peneira**: o paciente cai no primeiro teste que der True — os testes de baixo nem chegam a ser executados.

```
1 elif pressao > 12.0:
2     return "BAIXO"
3 else:
4     return "NORMAL"
```

O `else` não tem condição: ele é “tudo aquilo que sobrou”.

Por que sempre ter um `else`

Sem ele, um paciente que não casa com nenhum teste sai da função sem classificação — a função devolve `None` e o programa imprime Daniela: `None`.

Na prova

Uma saída `None` custa a questão inteira mesmo que toda a lógica esteja certa.


```
risco(15.0, 70)
```

#	Teste avaliado	Resultado	O que acontece
1	<code>15.0 < 9.0</code>	False	desce para o próximo
2	<code>15.0 > 14.0 and 70 > 60</code>	True and True → True	retorna ALTO
3	<code>15.0 > 14.0 or 70 > 60</code>	—	<i>nunca é executado</i>
4	<code>15.0 > 12.0</code>	—	<i>nunca é executado</i>
5	<code>else</code>	—	<i>nunca é executado</i>

A tabela de rastreio é a sua melhor ferramenta

Quando o programa dá uma resposta que você não esperava, monte esta tabela à mão. Ela mostra exatamente onde a decisão foi tomada.

A ordem importa

Experimento 1: trocar `elif` por `if`



```
1 def risco(pressao, idade):
2     classificacao = "NORMAL"
3     if pressao < 9.0:
4         classificacao = "CRITICO"
5     if pressao > 14.0 and idade > 60:
6         classificacao = "ALTO"
7     if pressao > 14.0 or idade > 60:
8         classificacao = "MEDIO"
9     if pressao > 12.0:
10        classificacao = "BAIXO"
11    return classificacao
```

As mesmas regras, os mesmos números, a mesma ordem.

Saída

Antonio: BAIXO
Beatriz: BAIXO
Cesar: BAIXO
Daniela: NORMAL
Eduardo: CRITICO

Três pacientes errados. O Antonio, que é ALTO, foi para a fila dos casos leves.

Com `if` soltos, **todos** os testes são executados — não existe peneira. Cada acerto sobrescreve o anterior.

#	Teste	Resultado	classificacao passa a valer
	—	—	"NORMAL"
1	15.0 < 9.0	False	"NORMAL"
2	>14.0 and >60	True	"ALTO"
3	>14.0 or >60	True	"MEDIO"
4	15.0 > 12.0	True	"BAIXO"

A lição

Com `if` independentes, quem manda é o **último** teste verdadeiro. Com `elif`, quem manda é o **primeiro**.

Experimento 2: manter o `elif`, trocar a ordem



```
1 def risco(pressao, idade):
2     if pressao > 12.0:
3         return "BAIXO"
4     elif pressao > 14.0 or idade > 60:
5         return "MEDIO"
6     elif pressao > 14.0 and idade > 60:
7         return "ALTO"
8     elif pressao < 9.0:
9         return "CRITICO"
10    else:
11        return "NORMAL"
```

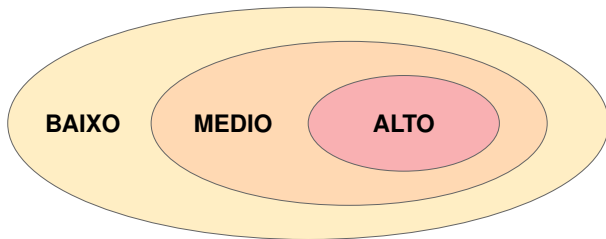
A peneira está lá. Só que na ordem errada.

Saída

Antonio: BAIXO
Beatriz: BAIXO
Cesar: BAIXO
Daniela: NORMAL
Eduardo: CRITICO

Repare

Dois erros **diferentes** produziram a **mesma** saída errada. Em ambos, o rótulo mais genérico ganhou do mais específico.



pacientes com pressão > 14.0 e idade > 60 cabem nos três conjuntos

A regra geral

Quando uma condição é um **caso particular** de outra, o caso particular tem de ser testado **primeiro**. Senão o teste mais largo captura o paciente antes.

mais específico → mais geral
↓

1. CRITICO — emergência: vale sozinho, ignora tudo

2. ALTO — dois agravantes ao mesmo tempo (and)

3. MEDIO — um agravante qualquer (or)

4. BAIXO — só um sinal leve

5. NORMAL — o que sobrou (else)

Como descobrir a ordem sozinho

Pergunte de cada par: “existe paciente que satisfaz as duas?” Se existir, a que descreve *menos* pacientes vem antes.

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

not True → False

not False → True

No enunciado da triagem

ALTO: pressão elevada **e** paciente idoso → and

MEDIO: pressão elevada **ou** paciente idoso → or

Armadilha frequente

```
1 if pressao > 14.0 and > 60:
```

Não existe. Cada lado do and precisa ser uma condição completa: `pressao > 14.0 and idade > 60`.

“Acima de 140” é > 14.0 . Não é ≥ 14.0 . Um caractere muda a resposta.

Pressão	Idade	Classificação	Por quê
14.0	70	MEDIO	140 <i>não</i> é acima de 140; mas 70 é acima de 60
14.0	30	BAIXO	nenhum agravante; 140 está acima de 120
12.0	20	NORMAL	120 <i>não</i> é acima de 120
9.0	45	NORMAL	90 <i>não</i> é inferior a 90
8.99	45	CRITICO	agora sim, abaixo de 90

Hábito de quem não perde ponto

Antes de entregar, teste os valores que ficam *exatamente* em cima do limite. É lá que o erro se esconde.

Sua vez

```
1 def risco(pressao, idade):  
2     if pressao > 12.0:  
3         return "BAIXO"  
4     elif pressao > 14.0 or idade > 60:  
5         return "MEDIO"  
6     elif pressao > 14.0 and idade > 60:  
7         return "ALTO"  
8     elif pressao < 9.0:  
9         return "CRITICO"  
10    else:  
11        return "NORMAL"
```

Sua tarefa

Reordene os testes — sem mudar nenhuma condição e sem acrescentar linha — para que a saída seja:

Antonio: ALTO
Beatriz: MEDIO
Cesar: MEDIO
Daniela: NORMAL
Eduardo: CRITICO

Antes de rodar

Diga em voz alta qual linha tem de subir e por quê.

Cenário

O protocolo mudou: pressão **de** 140 mmHg **para cima** (e não apenas acima de 140) já conta como pressão elevada. A idade continua “acima de 60”.

Sua tarefa

1. Faça a alteração no código.
2. **Antes de rodar**, preveja: *algum* dos cinco pacientes muda de classificação? Quais?
3. Rode e confira. Se errou a previsão, monte a tabela de rastreo desse paciente.

Para pensar

Quantos caracteres você mudou? E quantos pacientes isso afetaria num hospital com mil atendimentos por dia?

Cenário

A pediatria pediu uma sexta classificação: `PEDIATRICO` — paciente com **menos de 12 anos** e pressão acima de 120 mmHg. Quem está `CRITICO` continua `CRITICO`, em qualquer idade.

Sua tarefa

1. Em que posição da cascata o novo `elif` entra? Justifique antes de escrever.
2. Implemente e teste com (13.0, 8), (11.0, 8), (8.0, 5) e (13.0, 40).
3. Confira que os cinco pacientes originais não mudaram.

Critério de qualidade

Uma alteração bem feita não quebra o que já funcionava.

O que o programa faz

Lê N. Para cada paciente, lê nome, pressão e idade (uma por linha). Imprime <nome>: <classificacao>.

Formato exato

Dois-pontos, um espaço, classificação em maiúsculas. Nem um espaço a mais.

```
1 def risco(pressao, idade):  
2     # sua cascata de if / elif / else  
3  
4     n = int(input())  
5     for _ in range(n):  
6         nome = input()  
7         pressao = float(input())  
8         idade = int(input())  
9         print(nome + ": " + risco(pressao, idade))
```

Atenção aos tipos

`input()` devolve texto. Pressão é `float`, idade é `int`. Sem a conversão, comparar texto com número levanta `TypeError`.

Desafio

Escreva `classifica_chuva(mm, horas)` que classifica a precipitação de uma cidade:

- ENCHENTE: mais de 100 mm em menos de 6 horas
- ALERTA: mais de 100 mm (em qualquer tempo) **ou** mais de 50 mm em menos de 6 horas
- ATENCAO: mais de 50 mm
- NORMAL: os demais casos

O que se pede de verdade

Antes de programar, escreva no papel a **ordem** dos testes e aponte qual condição é caso particular de qual. O código é a parte fácil; a ordem é a parte que vale a nota.

Erro	Como evitar
Testar do mais geral para o mais específico	Comece pela emergência, termine no <code>else</code>
Usar <code>if</code> solto onde cabia <code>elif</code>	Se as respostas são exclusivas, use <code>elif</code>
Trocar <code>></code> por <code>>=</code> nos limites	Teste os valores exatamente em cima do limite
Esquecer o <code>else</code>	Sem ele a função devolve <code>None</code>
Esquecer <code>int()</code> / <code>float()</code> no <code>input()</code>	Comparar texto com número dá <code>TypeError</code>
Alterar o formato da saída	Copie o formato do enunciado, caractere a caractere

Os quatro fatos

1. Uma condição é uma expressão que vale `True` ou `False`.
2. `elif` garante que **exatamente uma** alternativa é escolhida — a primeira verdadeira.
3. Quando as descrições se sobrepõem, a **ordem** dos testes é que define a resposta: do mais específico ao mais geral.
4. Os erros moram nas fronteiras. Teste os limites.

Antes da próxima aula

Termine no notebook: a triagem com a categoria `PEDIATRICO` e a função `classifica_chuva`. Traga a sua tabela de rastreamento de um caso que você errou — vamos discutir em sala.



Allen B. Downey.

Think Python: How to Think Like a Computer Scientist.

O'Reilly Media, Sebastopol, 2 edition, 2016.



Eric Matthes.

Curso Intensivo de Python: Uma Introdução Prática e Baseada em Projetos à Programação.

Novatec, São Paulo, 3 edition, 2023.



Nilo Ney Coutinho Menezes.

Introdução à Programação com Python: Algoritmos e Lógica de Programação para Iniciantes.

Novatec, São Paulo, 3 edition, 2019.



Python Software Foundation.

The python tutorial: More control flow tools.

<https://docs.python.org/3/tutorial/controlflow.html>, 2026.

Documentação oficial da linguagem.



Sue Sentance, Jane Waite, and Maria Kallia.

Teaching computer programming with PRIMM: A sociocultural perspective.
Computer Science Education, 29(2-3):136–176, 2019.



Elliot Soloway.

Learning to program = learning to construct mechanisms and explanations.
Communications of the ACM, 29(9):850–858, 1986.